05-03 Program Runtime 분석

개요

Solana의 Program Runtime은 온체인 프로그램의 실행 환경을 제공하는 핵심 컴포넌트입니다. BPF (Berkeley Packet Filter) 기반의 가상 머신을 사용하여 스마트 컨트랙트를 안전하고 효율적으로 실행하며, 컴퓨트 예산 관리, 프로그램 로딩, 시스템 콜 처리 등의 기능을 담당합니다.

핵심 컴포넌트 구조

1. InvokeContext - 실행 컨텍스트

```
pub struct InvokeContext<'a> {
pub transaction_context: &'a mut TransactionContext,
sysvar_cache: &'a SysvarCache,
log_collector: Option<Rc<RefCell<LogCollector>>>,
compute_budget: ComputeBudget,
current_compute_budget: ComputeBudget,
compute_meter: RefCell<u64>,
pub programs_loaded_for_tx_batch: &'a LoadedProgramsForTxBatch,
pub programs_modified_by_tx: &'a mut LoadedProgramsForTxBatch,
pub feature_set: Arc<FeatureSet>,
pub timings: ExecuteDetailsTimings,
pub blockhash: Hash,
```

```
pub lamports_per_signature: u64,
pub syscall_context: Vec<Option<SyscallContext>>,
traces: Vec<Vec<[u64; 12]>>,
}
```

InvokeContext는 프로그램 실행 시 필요한 모든 컨텍스트 정보를 관리합니다:

- **transaction_context**: 트랜잭션 실행 컨텍스트
- **compute_budget**: 컴퓨트 리소스 제한
- **programs_loaded_for_tx_batch**: 로드된 프로그램 캐시
- **syscall_context**: 시스템 콜 실행 스택

InvokeContext 생성 및 초기화

```
impl<'a> InvokeContext<'a> {
pub fn new(
transaction_context: &'a mut TransactionContext,
sysvar_cache: &'a SysvarCache,
log_collector: Option<Rc<RefCell<LogCollector>>>,
compute_budget: ComputeBudget,
programs_loaded_for_tx_batch: &'a LoadedProgramsForTxBatch,
programs_modified_by_tx: &'a mut LoadedProgramsForTxBatch,
feature_set: Arc<FeatureSet>,
blockhash: Hash,
lamports_per_signature: u64,
```

```
) -> Self {
Self {
transaction_context,
sysvar_cache,
log_collector,
current_compute_budget: compute_budget,
compute_budget,
compute_meter: RefCell::new(compute_budget.compute_unit_limit),
programs_loaded_for_tx_batch,
programs_modified_by_tx,
feature_set,
timings: ExecuteDetailsTimings::default(),
blockhash,
lamports_per_signature,
syscall_context: Vec::new(),
traces: Vec::new(),
}
}
}
```

2. LoadedProgram - 프로그램 관리

```
pub struct LoadedProgram {
```

```
/// The program of this entry
pub program: LoadedProgramType,
/// Size of account that stores the program and program data
pub account_size: usize,
/// Slot in which the program was (re)deployed
pub deployment_slot: Slot,
/// Slot in which this entry will become active (can be in the future)
pub effective_slot: Slot,
/// How often this entry was used by a transaction
pub tx_usage_counter: AtomicU64,
/// How often this entry was used by an instruction
pub ix_usage_counter: AtomicU64,
/// Latest slot in which the entry was used
pub latest_access_slot: AtomicU64,
}
```

LoadedProgramType 열거형

```
pub enum LoadedProgramType {
/// Tombstone for programs which did not pass the verifier.
FailedVerification(ProgramRuntimeEnvironment),
/// Tombstone for programs which were explicitly undeployed / closed.
Closed,
/// Tombstone for programs which have recently been modified but the new version
```

```
DelayVisibility,

/// Successfully verified but not currently compiled.

Unloaded(ProgramRuntimeEnvironment),

/// Verified and compiled program of loader-v1 or loader-v2

LegacyV0(Executable<InvokeContext<'static>>),

/// Verified and compiled program of loader-v3 (aka upgradable loader)

LegacyV1(Executable<InvokeContext<'static>>),

/// Verified and compiled program of loader-v4

Typed(Executable<InvokeContext<'static>>),

/// A built-in program which is not stored on-chain but backed into and distribu

Builtin(BuiltinProgram<InvokeContext<'static>>),

}
```

프로그램 상태별 분류:

- **FailedVerification**: 검증 실패한 프로그램
- **Closed**: 명시적으로 닫힌 프로그램
- **DelayVisibility**: 수정되었지만 아직 활성화되지 않은 프로그램
- **Unloaded**: 검증되었지만 컴파일되지 않은 프로그램
- **LegacyV0/V1**: 레거시 로더 프로그램
- **Typed**: 새로운 loader-v4 프로그램
- **Builtin**: 내장 프로그램

컴퓨트 예산 관리

ComputeBudget 구조체

```
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub struct ComputeBudget {
/// Number of compute units that a transaction or individual instruction is allo
pub compute_unit_limit: u64,
/// Number of compute units consumed by a log_u64 call
pub log_64_units: u64,
/// Number of compute units consumed by a create_program_address call
pub create_program_address_units: u64,
/// Number of compute units consumed by an invoke call
pub invoke_units: u64,
/// Maximum program instruction invocation stack height
pub max_invoke_stack_height: usize,
/// Maximum cross-program invocation and instructions per transaction
pub max_instruction_trace_length: usize,
/// Base number of compute units consumed to call SHA256
pub sha256_base_cost: u64,
/// Incremental number of units consumed by SHA256 (based on bytes)
pub sha256_byte_cost: u64,
/// Maximum SBF to BPF call depth
pub max_call_depth: usize,
/// Size of a stack frame in bytes
```

```
pub stack_frame_size: usize,
/// program heap region size
pub heap_size: u32,
/// Number of compute units per additional 32k heap above the default
pub heap_cost: u64,
// ... 기타 비용 관련 필드들
}
```

컴퓨트 단위 소비 추적

```
impl<'a> ContextObject for InvokeContext<'a> {
fn consume(&mut self, amount: u64) {
// 1 to 1 instruction to compute unit mapping
// ignore overflow, Ebpf will bail if exceeded
let mut compute_meter = self.compute_meter.borrow_mut();
*compute_meter = compute_meter.saturating_sub(amount);
}

fn get_remaining(&self) -> u64 {
*self.compute_meter.borrow()
}
}
```

프로그램 실행 프로세스

1. MessageProcessor - 메시지 처리

```
impl MessageProcessor {
pub fn process_message(
message: &SanitizedMessage,
program_indices: &[Vec<IndexOfAccount>],
transaction_context: &mut TransactionContext,
log_collector: Option<Rc<RefCell<LogCollector>>>,
programs_loaded_for_tx_batch: &LoadedProgramsForTxBatch,
programs_modified_by_tx: &mut LoadedProgramsForTxBatch,
feature_set: Arc<FeatureSet>,
compute_budget: ComputeBudget,
timings: &mut ExecuteTimings,
sysvar_cache: &SysvarCache,
blockhash: Hash,
lamports_per_signature: u64,
accumulated_consumed_units: &mut u64,
) -> Result<(), TransactionError> {
let mut invoke_context = InvokeContext::new(
transaction_context,
sysvar_cache,
```

```
log_collector,

compute_budget,

programs_loaded_for_tx_batch,

programs_modified_by_tx,

feature_set,

blockhash,

lamports_per_signature,

);



// 각 인스트럭션 처리

for (instruction_index, ((program_id, instruction), program_indices)) in  messag

.program_instructions_iter()

.zip(program_indices.iter())

.enumerate()

{

// 프리컴파일 프로그램 확인

let  is_precompile = is_precompile(program_id, |id| invoke_context.feature_set.i

// 인스트럭션 실행

// ...

}

}

}
```

2. 프로그램 실행 단계

1. **프로그램 로드**: 캐시에서 프로그램 검색 또는 새로 로드
2. **컨텍스트 설정**: InvokeContext 초기화
3. **리소스 검증**: 컴퓨트 예산 및 스택 깊이 확인
4. **실행**: BPF 가상 머신에서 프로그램 실행
5. **결과 처리**: 계정 상태 업데이트 및 로그 수집

3. 프로그램 캐시 관리

```
impl<'a> InvokeContext<'a> {
pub  fn  find_program_in_cache(&self, pubkey: &Pubkey) -> Option<Arc<LoadedProgr
// First lookup the cache of the programs modified by the current transaction
self.programs_modified_by_tx
.find(pubkey)
.or_else(|| self.programs_loaded_for_tx_batch.find(pubkey))
}
}
```

BPF 프로그램 로더

1. 프로그램 로더 유형

Solana는 여러 세대의 프로그램 로더를 지원합니다:

- **Native Loader**: 내장 프로그램용

- **BPF Loader (Deprecated)**: 초기 BPF 프로그램
- **BPF Loader**: 표준 BPF 프로그램
- **BPF Loader Upgradeable**: 업그레이드 가능한 프로그램
- **Loader v4**: 최신 프로그램 로더

2. 프로그램 검증 및 컴파일

```
pub type ProgramRuntimeEnvironment = Arc<BuiltinProgram<InvokeContext<'static>>
```

프로그램 로딩 과정:

1. **바이트코드 검증**: BPF 바이트코드의 유효성 검사
2. **JIT 컴파일**: 네이티브 코드로 컴파일 (선택적)
3. **캐시 저장**: 컴파일된 프로그램을 캐시에 저장
4. **실행 준비**: 실행 환경 설정

시스템 콜 처리

1. 내장 함수 매크로

```
#[macro_export]
macro_rules! declare_process_instruction {
($process_instruction:ident, $cu_to_consume:expr, |$invoke_context:ident| $inner
$crate::solana_rbpf::declare_builtin_function!(
$process_instruction,
fn rust(
```

```
invoke_context: &mut $crate::invoke_context::InvokeContext,

_arg0: u64, _arg1: u64, _arg2: u64, _arg3: u64, _arg4: u64,

_memory_mapping: &mut $crate::solana_rbpf::memory_region::MemoryMapping,

) -> std::result::Result<u64, Box<dyn  std::error::Error>> {

// 컴퓨트 단위 소비

let  consumption_result = if $cu_to_consume > 0 {

invoke_context.consume_checked($cu_to_consume)

} else {

Ok(())

};

consumption_result.and_then(|_| {

process_instruction_inner(invoke_context)

.map(|_| 0)

.map_err(|err| Box::new(err) as Box<dyn  std::error::Error>)

})

}

);

};

}
```

2. 시스템 콜 컨텍스트

```
pub  syscall_context: Vec<Option<SyscallContext>>,
```

시스템 콜 실행 시:

- 스택 프레임 관리
- 컴퓨트 단위 소비
- 메모리 접근 제어
- 오류 처리

성능 최적화

1. 프로그램 캐시 통계

```
#[derive(Debug, Default)]
pub struct Stats {
/// a program was already in the cache
pub hits: AtomicU64,
/// a program was not found and loaded instead
pub misses: AtomicU64,
/// a compiled executable was unloaded
pub evictions: HashMap<Pubkey, u64>,
/// an unloaded program was loaded again
pub reloads: AtomicU64,
/// a program was loaded or un/re/deployed
pub insertions: AtomicU64,
// ... 기타 통계 필드들
```

```
}
```

2. 최적화 기법

- **JIT 컴파일**: 런타임에 네이티브 코드로 컴파일
- **프로그램 캐싱**: 자주 사용되는 프로그램을 메모리에 캐시
- **지연 로딩**: 필요한 시점에만 프로그램 로드
- **사용량 추적**: 프로그램 사용 빈도 기반 캐시 관리

보안 및 안전성

1. 리소스 제한

- **컴퓨트 예산**: 무한 루프 방지
- **스택 깊이 제한**: 스택 오버플로우 방지
- **메모리 제한**: 힙 크기 제한
- **호출 깊이 제한**: 재귀 호출 제한

2. 메모리 안전성

- **메모리 매핑**: 안전한 메모리 접근
- **경계 검사**: 배열 접근 시 경계 검사
- **타입 안전성**: 강타입 시스템

요약

Program Runtime은 Solana의 스마트 컨트랙트 실행을 담당하는 핵심 시스템입니다:

- **InvokeContext**: 프로그램 실행 컨텍스트 관리
- **LoadedProgram**: 프로그램 로딩 및 캐싱
- **ComputeBudget**: 리소스 사용량 제한 및 추적
- **MessageProcessor**: 트랜잭션 메시지 처리
- **BPF 가상 머신**: 안전한 프로그램 실행 환경
- **최적화**: JIT 컴파일, 캐싱, 지연 로딩

이러한 설계를 통해 Solana는 높은 성능과 보안성을 동시에 제공하며, 복잡한 스마트 컨트랙트를 효율적으로 실행할 수 있습니다.